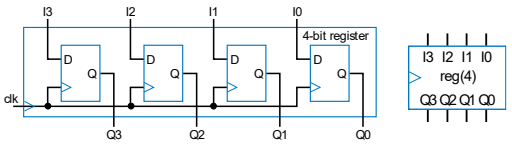
Basic Register

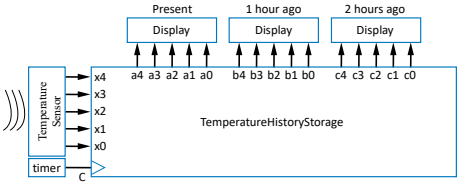
- Typically, we store multi-bit items
  - e.g., storing a 4-bit binary number
- ***Register***: multiple flip-flops sharing clock signal
  - From this point, we'll use registers for bit storage
    - No need to think of latches or flip-flops
    - But now you know what's inside a register



Digital Design
Copyright © 2006
Frank Vahid

Example Using Registers: Temperature Display

- Temperature history display
  - Sensor outputs temperature as 5-bit binary number
  - Timer pulses C every hour
  - Record temperature on each pulse, display last three recorded values

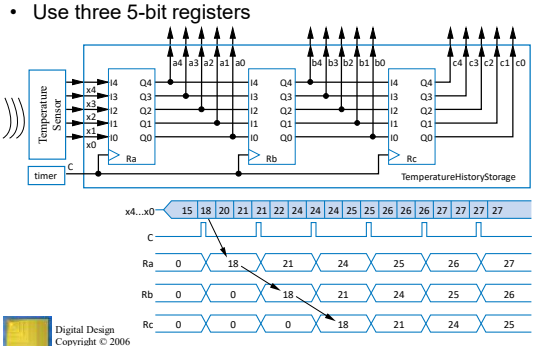


(In practice, we would actually avoid connecting the timer output C to a clock input, instead only connecting an oscillator output to a clock input.)

Digital Design
Copyright © 2006
Frank Vahid

Example Using Registers: Temperature Display

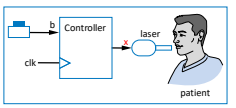
- Use three 5-bit registers



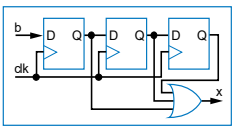
Digital Design
Copyright © 2006
Frank Vahid

Finite-State Machines (FSMs) and Controllers

- Want sequential circuit with particular behavior over time
  - **State** of the circuit
  - State can be intentionally used to design circuits that have a specific behavior over time.



- **Example: Laser timer**
  - Push button: x=1 for 3 clock cycles
  - How? Let's try three flip-flops
    - b=1 gets stored in first D flip-flop
    - Then 2nd flip-flop on next cycle, then 3rd flip-flop on next
    - OR the three flip-flop outputs, so x should be 1 for three cycles



Three-cycles-high laser timer - a poorly done first design

Digital Design
Copyright © 2006
Frank Vahid

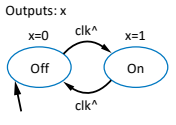
Need a Better Way to Design Sequential Circuits

- **Trial and error** is not a good design method
  - Will we be able to "guess" a circuit that works for other desired behavior?
    - How about counting up from 1 to 9? Pulsing an output for 1 cycle every 10 cycles? Detecting the sequence 1 3 5 in binary on a 3-bit input?
  - And, a circuit built by guessing may have undesired behavior
    - Laser timer: What if press button again while x=1? x then stays one another 3 cycles. Is that what we want?
- **Combinational circuit design** process had two important things
  1. A formal way to describe desired circuit behavior
     - Boolean equation, or truth table
  2. A well-defined process to convert that behavior to a circuit
- **We need those things for sequence circuit design**

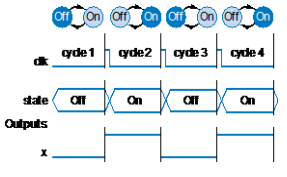
Digital Design
Copyright © 2006
Frank Vahid

Describing Behavior of Sequential Circuit: FSM

- **Finite-State Machine (FSM)**
  - A way to describe desired behavior of sequential circuit
    - Akin to Boolean equations for combinational behavior
  - List all possible states, and transitions among states



- **Example: Make x change toggle (0 to 1, or 1 to 0) every clock cycle**
  - Two states: "Off" (x=0), and "On" (x=1)
  - Transition from Off to On, or On to Off, on rising clock edge
  - Arrow with no starting state points to initial state (when circuit first starts)



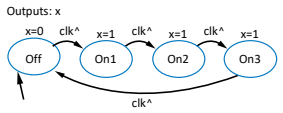
While representing multibit data, a vertical line (or a pair of crossed lines) shows when the values change.

Digital Design
Copyright © 2006
Frank Vahid

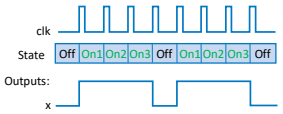
FSM Example: 0,1,1,1,repeat

Example: Build a system that held its output high for three cycles, return to 0 and repeats

- **Want 0, 1, 1, 1, 0, 1, 1, 1, ...**
  - Each value for one clock cycle
- Can describe as FSM
  - Four states
  - Transition on rising clock edge to next state



The state names in state machine could have been *S0, S1, S2,* and *S3,* but names that describe a state's purpose in the FSM are preferable.

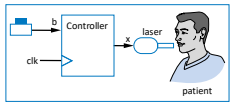


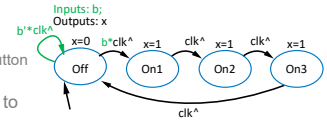
NEXT ➔ Extend FSM to Three-Cycles High Laser Timer

30

Extend FSM to Three-Cycles High Laser Timer

- Four states
- Wait in "Off" state while b is 0 (b′)
- When b is 1 (and rising clock edge), transition to On1
  - Sets x=1
- On next two clock edges, transition to On2, then On3
  - also set x=1 in each state
  - So x=1 for three cycles after button pressed
- On next clock edge, transition to Off
  - Sets x=0



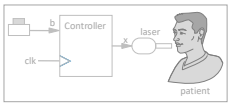


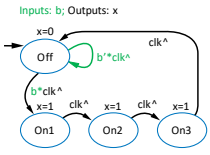
Boolean expressions can be associated with the transitions to extend the behavior

31

Extend FSM to Three-Cycles High Laser Timer

- Four states
- Wait in "Off" state while b is 0 (b′)
- When b is 1 (and rising clock edge), transition to On1
  - Sets x=1
- On next two clock edges, transition to On2, then On3
  - also set x=1 in each state
  - So x=1 for three cycles after button pressed
- On next clock edge, transition to Off
  - Sets x=0



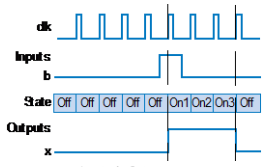


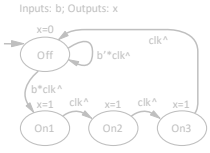
Boolean expressions can be associated with the transitions to extend the behavior

32

Extend FSM to Three-Cycles High Laser Timer

- Four states
- Wait in "Off" state while b is 0 (b′)
- When b is 1 (and rising clock edge), transition to On1
  - Sets x=1
- On next two clock edges, transition to On2, then On3
  - also set x=1 in each state
  - So x=1 for three cycles after button pressed
- On next clock edge, transition to Off
  - Sets x=0



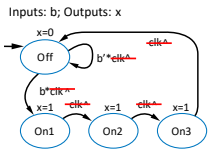


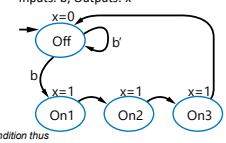
Boolean expressions can be associated with the transitions to extend the behavior

33

FSM Simplification: Rising Clock Edges Implicit

- Showing rising clock on every transition: cluttered
  - Make **implicit** -- assume every edge has rising clock, even if not shown
  - What if we wanted a transition *without* a rising edge
    - We don't consider such asynchronous FSMs -- less common, and advanced topic
    - Only consider ***synchronous*** FSMs -- rising edge on *every* transition



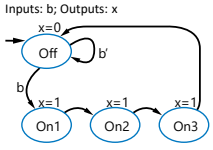


Note: Transition with no associated condition thus transistions to next state on next clock cycle

34

FSM Definition

- **FSM consists of**
  - Set of states
    - Ex: {Off, On1, On2, On3}
  - Set of inputs, set of outputs
    - Ex: Inputs: {b}, Outputs: {x}
  - Initial state
    - Ex: "Off"
  - Set of transitions
    - Directed edges with associated input conditions to indicate next states
    - Ex: Has 5 transitions
  - Set of actions
    - Sets outputs while in states
    - Ex: x=0, x=1, x=1, and x=1



We often draw FSM graphically, known as ***state diagram***

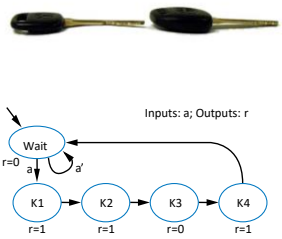
Can also use table (state table), or textual languages

Assigning an output in an FSM is known as an *action*.

35

FSM Example: Secure Car Key

- Many new car keys include tiny computer chip
  - When car starts, car's computer (under engine hood) requests identifier from key
  - Key transmits identifier
    - If not, computer shuts off car
- FSM for chip in the key
  - Wait until computer requests ID (a=1)
  - Transmit ID
    - in this exemplary case, 1101

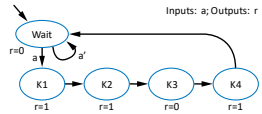


LSB ➔ MSB

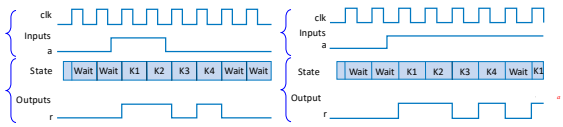
36

FSM Example: Secure Car Key (cont.)

- Nice feature of FSM
  - Executing or Running an FSM
    - Can evaluate output behavior for different input sequence
  - Timing diagrams represents
    - represent a particular situation defined by how we set the inputs.
    - show states and output values for different input waveforms

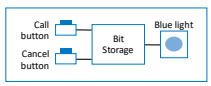


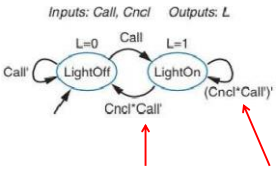
Q: Determine states and r value for given input waveform:

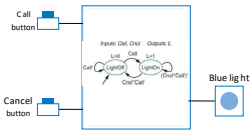


37

Flight-attendant call button







Only condition to transition back
F = Cncl * Call'
= m2
1,0

Rest of the conditions for no change
F' = (Cncl * Call')'
= Cncl' + Call
= m0 + m1 + m3
0,0
0,1
1,1

38

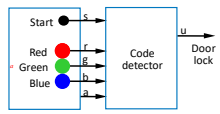
Capturing Desired System Behavior as an FSM

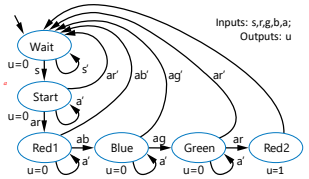
- A challenging task for a designer
- Following method can help
  - List States
  - Create transitions
  - Refine the FSM
- The method described above is just a guide.
- Capturing behavior as an FSM may require some creativity and trial-and-error
- Many different FSMs could be created that describe the same desired behavior
  - one FSM may be easier to understand while another FSM may have fewer states
- Experience can help greatly in creating correct and easy-to-understand FSMs that capture desired system behavior

39

FSM Example: Code Detector

- Unlock door (u=1) only when buttons pressed in sequence:
  - start, then red, blue, green, red
- Input from each button: *s, r, g, b*
  - Also, output *a* indicates that some colored button pressed
- FSM
  - Wait for start (s=1) in "Wait"
  - Once started ("Start")
    - If see red, go to "Red1"
    - Then, if see blue, go to "Blue"
    - Then, if see green, go to "Green"
    - Then, if see red, go to "Red2"
      - In that state, open the door (u=1)
    - Wrong button at any step, return to "Wait", without opening door



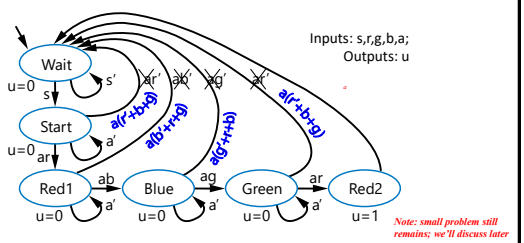


Q: Can you trick this FSM to open the door, without knowing the code?

A: Yes, hold all buttons simultaneously

40

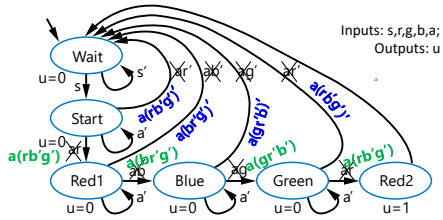
Improve FSM for Code Detector (cont.)



- New transition conditions detect if wrong button pressed, returns to "Wait"
- FSM provides formal, concrete means to accurately define desired behavior

41

Improve FSM for Code Detector (cont.)



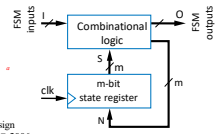
- New transition conditions enables only one transition
- New transition conditions detect if wrong button pressed, returns to "Wait"
- FSM provides formal, concrete means to accurately define desired behavior

Digital Design
Copyright © 2006
Frank Vahid

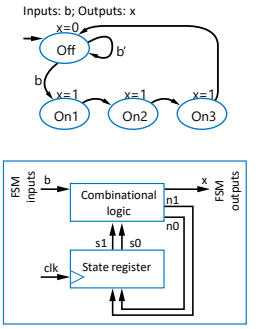
42

Standard Controller Architecture

- How implement FSM as sequential circuit?
  - Use standard architecture
    - State register -- to store the present state
    - Combinational logic -- to compute outputs, and next state
    - For laser timer FSM
      - 2-bit state register, can represent four states
      - Input b, output x
  - Known as ***controller***



General version



Digital Design
Copyright © 2006
Frank Vahid

43

3.4

Controller Design

- Controller Design Process — Capture & Convert

TABLE 3.2 Controller design process.

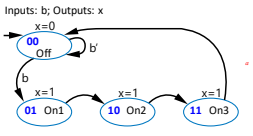
Step		Description
Step 1: Capture behavior	*Capture the FSM*	Create an FSM that describes the desired behavior of the controller.
Step 2: Convert to circuit — 2A	*Set up the architecture*	Set up the standard architecture by using a state register of appropriate width and combinational logic. The logic's inputs are the state register bits and the FSM inputs; the logic's outputs are the next state bits and the FSM outputs.
2B	*Encode the states*	Assign a unique binary number, known as an ***encoding***, to each state. Any encoding is sufficient as long as each state has a unique encoding. Usually a minimum number of bits is used and an encoding is assigned to each state by counting up in binary.
2C	*Fill in the truth table*	Translate the FSM into a truth table for the combinational logic such that the logic will generate the outputs and next state signals for the given FSM. Ordering the inputs with state bits first makes the correspondence between the table and the FSM clear.
2D	*Implement the combinational logic*	Implement the combinational logic using any method.

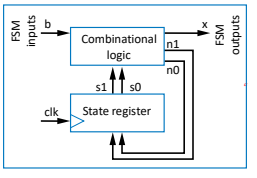
Digital Design
Copyright © 2006
Frank Vahid

44

Controller Design: Laser Timer Example

- Step 1: Capture the FSM
  - Already done
- Step 2A: Set up the architecture
  - 2-bit state register (for 4 states)
  - Input b, output x
  - Next state signals n1, n0
- Step 2B: Encode the states
  - Any encoding with each state unique will work





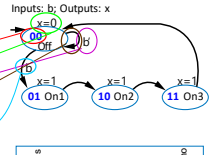
Digital Design
Copyright © 2006
Frank Vahid

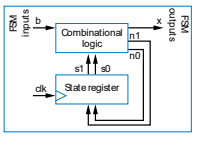
45

Controller Design: Laser Timer Example (cont.)

- Step 2C: Fill in the truth table

	Inputs			Outputs		
	s1	s0	b	x	n1	n0
Off	0	0	0	0	0	0
	0	0	1	0	0	1
On1	0	1	0	1	1	0
	0	1	1	1	1	0
On2	1	0	0	1	1	1
	1	0	1	1	1	1
On3	1	1	0	1	0	0
	1	1	1	1	0	0





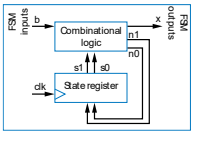
Digital Design
Copyright © 2006
Frank Vahid

46

Controller Design: Laser Timer Example (cont.)

- Step 2D: Implement the combinational logic

	Inputs			Outputs		
	s1	s0	b	x	n1	n0
Off	0	0	0	0	0	0
	0	0	1	0	0	1
On1	0	1	0	1	1	0
	0	1	1	1	1	0
On2	1	0	0	1	1	1
	1	0	1	1	1	1
On3	1	1	0	1	0	0
	1	1	1	1	0	0



x = s1 + s0 (note from the table that x=1 if s1 = 1 or s0 = 1)

n1 = s1's0b' + s1's0b + s1s0'b' + s1s0'b

n1 = s1's0 + s1s0'

n0 = s1's0'b + s1s0'b' + s1s0'b

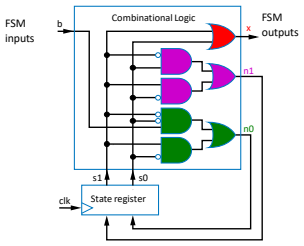
n0 = s1's0'b + s1s0'

Digital Design
Copyright © 2006
Frank Vahid

47

Controller Design: Laser Timer Example (cont.)

- Step 2D: Implement the combinational logic (cont.)

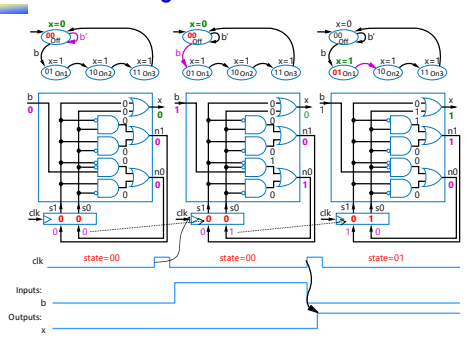


	Inputs			Outputs		
	s1	s0	b	x	n1	n0
Off	0	0	0	0	0	0
	0	0	1	0	0	1
On1	0	1	0	1	1	0
	0	1	1	1	1	0
On2	1	0	0	1	1	1
	1	0	1	1	1	1
On3	1	1	0	1	0	0
	1	1	1	1	0	0

x = s1 + s0
n1 = s1's0 + s1s0'
n0 = s1's0'b + s1s0'

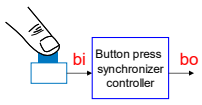
48

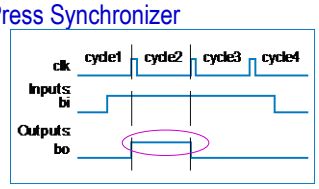
Understanding the Controller's Behavior



49

Controller Example: Button Press Synchronizer



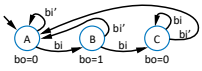


- Want simple sequential circuit that converts button press to single cycle duration, regardless of length of time that button actually pressed
  - We assumed such an ideal button press signal in earlier example, like the button in the laser timer controller

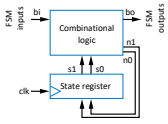
50

Controller Example: Button Press Synchronizer (cont.)

FSM inputs: bi; FSM outputs: bo

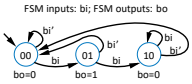


Step 1: Capture the FSM



Step 2A: Set up the architecture

FSM inputs: bi; FSM outputs: bo

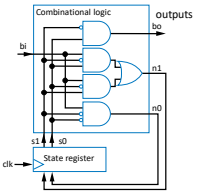


Step 2B: Encode the states

Combinational logic							
	Inputs			Outputs			
	s1	s0	bi	n1	n0	bo	
A	0	0	0	0	0	0	
	0	0	1	0	1	0	
B	0	1	0	0	0	1	
	0	1	1	1	0	1	
C	1	0	0	0	0	0	
	1	0	1	1	0	0	
unused	1	1	0	0	0	0	
	1	1	1	0	0	0	

Step 2C: Fill in the truth table

n1 = s1's0bi + s1s0bi
n0 = s1's0'bi
bo = s1's0bi' + s1's0bi = s1s0



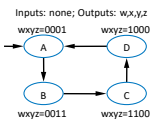
Step 2D: Implement the combinational logic

51

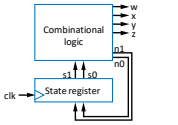
Controller Example: Sequence Generator

- Want generate sequence 0001, 0011, 1100, 1000, (repeat)
  - Each value for one clock cycle
  - Common, e.g., to create pattern in 4 lights, or control magnets of a "stepper motor"

Inputs: none; Outputs: w,x,y,z

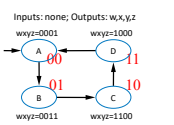


Step 1: Capture the FSM



Step 2A: Set up the architecture

Inputs: none; Outputs: w,x,y,z

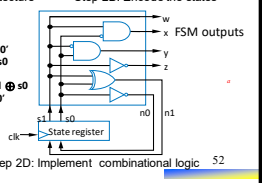


Step 2B: Encode the states

	Inputs		Outputs					
	s1	s0	w	x	y	z	n1	n0
A	0	0	0	0	0	1	0	1
B	0	1	0	0	1	1	1	0
C	1	0	1	1	0	0	1	1
D	1	1	1	0	0	0	0	0

Step 2C: Fill in the truth table

w = s1
x = s1s0'
y = s1's0
z = s1'
n1 = s1 ⊕ s0
n0 = s0'

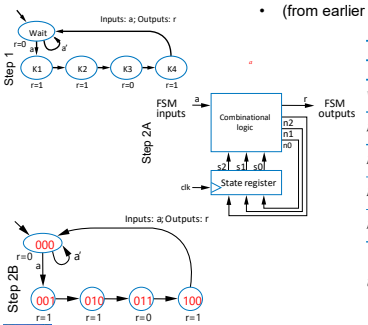


Step 2D: Implement combinational logic

52

Controller Example: Secure Car Key

- (from earlier example)



	Inputs				Outputs			
	s2	s1	s0	a	r	n2	n1	n0
Wait	0	0	0	0	0	0	0	0
	0	0	0	1	0	0	0	1
K1	0	0	1	0	1	0	1	0
	0	0	1	1	1	0	1	0
K2	0	1	0	0	1	0	1	1
	0	1	0	1	1	0	1	1
K3	0	1	1	0	0	1	0	0
	0	1	1	1	0	1	0	0
K4	1	0	0	0	1	0	0	0
	1	0	0	1	1	0	0	0
Unused	1	0	1	0	0	0	0	0
	1	0	1	1	0	0	0	0
	1	1	0	0	0	0	0	0
	1	1	0	1	0	0	0	0
	1	1	1	0	0	0	0	0
	1	1	1	1	0	0	0	0

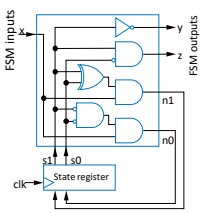
Step 2C

Do yourself!
Step 2D: Implement the combinational logic

53

Example: Seq. Circuit to FSM (Reverse Engineering)

What does this circuit do?



Work backwards

Step 2D

Step 2C

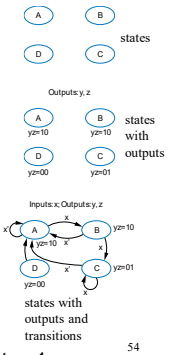
y=s1'
z = s1s0'
n1=(s1 xor s0)x
n0=(s1'*s0')x

	Inputs			Outputs			
	s1	s0	x	n1	n0	y	z
A	0	0	0	0	0	1	0
	0	0	1	0	1	1	0
B	0	1	0	0	0	1	0
	0	1	1	1	0	1	0
C	1	0	0	0	0	0	1
	1	0	1	1	0	0	1
D	1	1	0	0	0	0	0
	1	1	1	0	0	0	0

Pick any state names you want

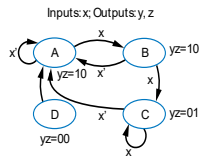
Step 2B

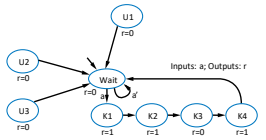
Step 2A ➔ Do Yourself!



Step 1

Representing and Handling of Unused States





Common Pitfalls Regarding Transition Properties

- *Only* one condition **should** be true
  - For all transitions leaving a state
  - Else, which one?
- *One* condition **must** be true
  - For all transitions leaving a state
  - Else, where go?

ab=11 – next state?

Mutually Exclusive Transitions
— Exclusive Transitions Property

what if ab=00?

All Transitions
— Complete Transitions Property

Verifying Correct Transition Properties using Boolean Algebra

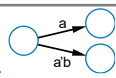
- Only one condition true — Exclusive Transitions Property:
  AND of each condition pair (for transitions leaving a state) should equal 0
  → proves pair can never simultaneously be true
- One condition must be true — Complete Transitions Property:
  OR of all conditions of transitions leaving a state) should equal 1
  → proves at least one condition must be true

Answer:

a * a'b
= (a * a') * b
= 0 * b
= 0
OK!

a + a'b
= a*(1+b) + a'b
= a + ab + a'b
= a + (a+a')b
= a + b
Fails! Might not be 1 (i.e., a=0, b=0)

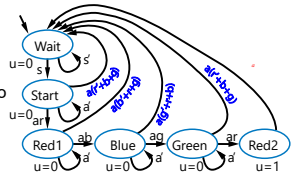
- **Example**



Q: For shown transitions, prove whether:
* Only one condition true (AND of each pair is always 0)
* One condition true (OR of all transitions is always 1)

Evidence that Pitfall is Common

- Recall code detector FSM
  - We "fixed" a problem with the transition conditions
  - Do the transitions obey the two required transition properties?
    - Consider transitions of state *Start*, and the "only one true" property



ar * a'
= (a*a')r
= 0*r
= 0

a' * a(r'+b+g)
= (a'*a)*(r'+b+g)
= 0*(r'+b+g)
= 0

ar * a(r'+b+g)
= (a*a)*r*(r'+b+g)
= a*r*(r'+b+g)
= arr'+arb+arg
= 0 + arb+arg
= arb + arg
= ar(b+g)

Fails! Means that two of Start's transitions could be true

Intuitively: press red and blue buttons at same time: conditions ar, and a(r'+b+g) will both be true. Which one should be taken?

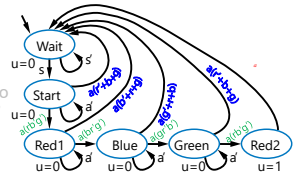
Q: How to solve?

A: ar should be arb'g' (likewise for ab, ag, ar)

Note: As evidence the pitfall is common, we admit the mistake was not intentional. A reviewer of the book caught it.

Evidence that Pitfall is Common (cont.)

- Recall code detector FSM
  - We "fixed" a problem with the transition conditions
  - Do the transitions obey the two required transition properties?
    - Consider transitions of state *Start*, and the "only one true" property



arb'g' * a'
= aa' * rb'g'
= 0*rb'g
= 0
OK

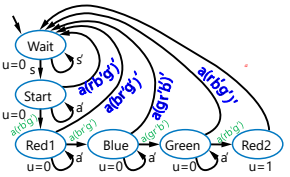
a' * a(r'+b+g)
= a' * a(rb'g')'
= 0 * (rb'g')'
= 0
OK

arb'g' * a(r'+b+g)
= arb'g' * a(rb'g')'
= (a*a)*(rb'g')*(rb'g')'
write rb'g' as Y for clarity ...
= (a*a)*Y*Y'
= (a*a)*0
= 0
OK

Intuitively: press red and blue buttons at same time: conditions a(rb'g') will be false and a(r'+b+g) will be true.

Evidence that Pitfall is Common (cont.)

- Recall code detector FSM
  - We "fixed" a problem with the transition conditions
  - Do the transitions obey the two required transition properties?
    - Consider transitions of state *Start*, and the "only one true" property



arb'g' * a'
= aa' * rb'g'
= 0*rb'g'
= 0
OK

a' * a(rb'g')'
= 0 * (rb'g')'
= 0
OK

arb'g' * a(rb'g')'
= (a*a)*(rb'g')*(rb'g')'
write rb'g' as Y for clarity ...
= (a*a)*Y*Y'
= (a*a)*0
= 0
OK

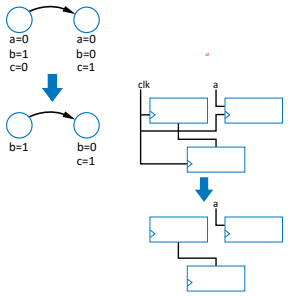
Intuitively: press red and blue buttons at same time: conditions a(rb'g') will be false and a(rb'g')' will be true.

Its is less error prone to use X expression on one transition and X' on other transition when to ensure exclusivity. For example, on Start's transitions, one condition is a(rb'g') and other condition is a(rb'g')' i.e., a*X and a*X'

Digital Design
Copyright © 2006
Frank Vahid

Simplifying Notations

- FSMs
  - Assume unassigned output implicitly assigned 0
- Sequential circuits
  - Assume unconnected clock inputs connected to same external clock



Digital Design
Copyright © 2006
Frank Vahid

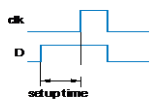
More on Flip-Flops and Controllers

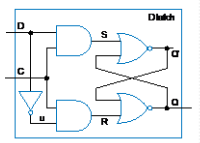
- Other flip-flop types
  - SR flip-flop: like SR latch, but edge triggered
  - JK flip-flop: like SR (S→J, R→K)
    - But when JK=11, toggles
    - 1→0, 0→1
  - T flip-flop: JK with inputs tied together
    - Toggles on every rising clock edge
  - Previously utilized to minimize logic outside flip-flop
    - Today, minimizing logic to such extent is not as important
    - D flip-flops are thus by far the most common

Digital Design
Copyright © 2006
Frank Vahid

Non-Ideal Flip-Flop Behavior

- Can't change flip-flop input too close to clock edge
  - **Setup time**: time that D must be stable *before* edge
    - Else, stable value not present at internal latch
  - **Hold time**: time that D must be held stable *after* edge
    - Else, new value doesn't have time to loop around and stabilize in internal latch



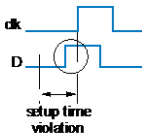


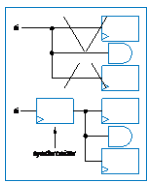
A race condition:
Leads to oscillation!

Digital Design
Copyright © 2006
Frank Vahid

Metastability

- Violating setup/hold time can lead to bad situation known as ***metastable*** state
  - Metastable state: Any flip-flop state other than stable 1 or 0
    - Eventually settles to one or other, but we don't know which
  - For internal circuits, we can make sure observe setup time
  - But what if input comes from external (asynchronous) source, e.g., button press?
- Partial solution
  - Insert synchronizer flip-flop for asynchronous input
    - Special flip-flop with very small setup/hold time
  - Doesn't completely prevent metastability



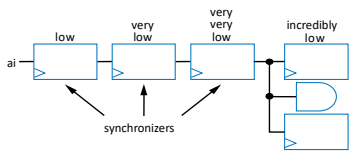


Digital Design
Copyright © 2006
Frank Vahid

Metastability

- One flip-flop doesn't completely solve problem
- How about adding more synchronizer flip-flops?
  - Helps, but just decreases probability of metastability
- So how solve completely?
  - Can't! May be unsettling to new designers. But we just can't guarantee a design that won't ever be metastable. We can just minimize the mean time between failure (MTBF) -- a number often given along with a circuit

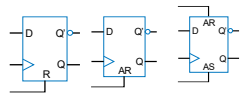
Probability of flip-flop being metastable is...

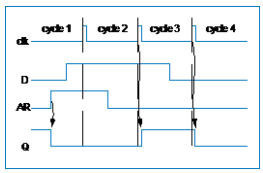


Digital Design
Copyright © 2006
Frank Vahid

Flip-Flop Set and Reset Inputs

- Some flip-flops have additional inputs
  - Synchronous reset:
    - clears Q to 0 on next clock edge
  - Synchronous set:
    - sets Q to 1 on next clock edge
  - Asynchronous reset:
    - clear Q to 0 immediately (not dependent on clock edge)
    - Example timing diagram shown
  - Asynchronous set:
    - set Q to 1 immediately

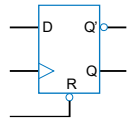




Example — Asynchronous reset timing diagram

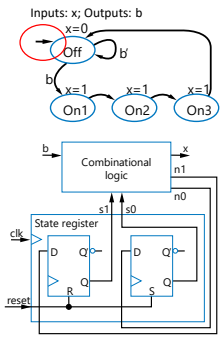
Active Low Inputs

- We've assumed input action occur when input is 1
  - Some inputs are instead active when input is 0 -- "active low"
  - Shown with inversion bubble
  - So to reset the shown flip-flop, set R=0. Else, keep R=1.



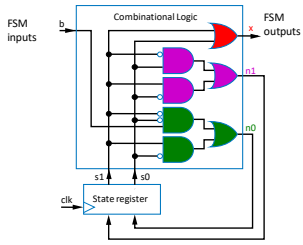
Initial State of a Controller

- All our FSMs had initial state
  - But our sequential circuit designs did not
  - Can accomplish using flip-flops with reset/set inputs
    - Shown circuit initializes flip-flops to 01
  - Designer must ensure reset input is 1 during power up of circuit
    - By electronic circuit design



Glitching

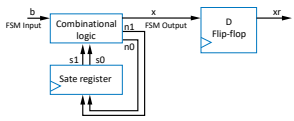
- Glitch: Temporary values on outputs that appear soon after input changes, before stable new output values
- Designer must determine whether glitching outputs may pose a problem
  - If so, may consider adding flip-flops to outputs
    - Delays output by one clock cycle, but may be OK



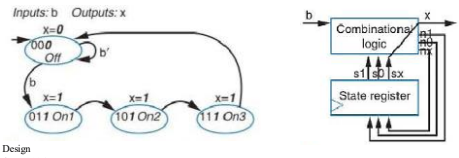
Example — Three-cycles-high laser timer design

Glitching Solutions and their Overheads

- Registered Outputs



- State variable as Outputs



Product Profile — Pacemaker

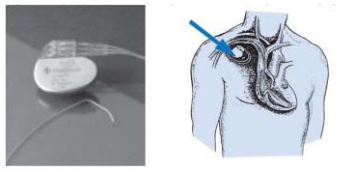


Figure 3.91 Pacemaker with leads (left), and pacemaker's location under the skin (right). Courtesy of Medtronic, Inc.

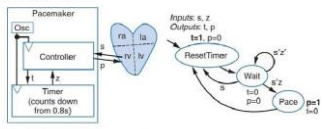


Figure 3.92 A basic pacemaker's controller FSM.

Chapter Summary

- Sequential circuits
  - Have state
- Created robust bit-storage device: D flip-flop
  - Put several together to build register, which we used to hold state
- Defined FSM formal model to describe sequential behavior
  - Using solid mathematical models -- Boolean equations for combinational circuit, and FSMs for sequential circuits -- is very important.
- Defined 5-step process to convert FSM to sequential circuit
  - Controller
- So now we know how to build the class of sequential circuits known as controllers

Digital Design
Copyright © 2006
Frank Vahid